INNOMATICS RESEARCH LABS

Task 2: Python Problem-Solving Practice

Objective

The objective of this task is to improve your ability to **understand problems, develop logical solutions, and implement them using Python in Jupyter Notebook**.

These exercises will help you strengthen your problem-solving skills, improve your understanding of Python fundamentals, and build confidence in coding assessments, competitive programming screenings, and technical interviews.

Problem-Solving Approach

Follow these six steps for every question:

Observe → Understand → Break → Logic → Code → Test

- **Observe:** Identify the given input and expected output.
- **Understand:** Determine exactly what the problem is asking.
- **Break:** Divide the problem into smaller, manageable steps.
- **Logic:** Develop the solution approach in simple English or pseudocode.
- **Code:** Implement your logic using Python.
- **Test:** Execute the code and verify the output with different inputs.

Important Rules

1. Read and understand each problem carefully before writing code.
2. Do not search for the solution before making your own attempt.
3. Follow the six-step problem-solving approach for every question.
4. Write your logic in simple English or pseudocode before coding.
5. Write and execute your Python code in Jupyter Notebook.
6. Do not hardcode the expected output.
7. Test your solution using the given example and at least one additional test case.
8. Handle edge cases wherever applicable.

9. Keep the original question number clearly visible.
10. Complete all **20 questions** independently.
11. Do not copy solutions or explanations.
12. Focus on understanding **why and how your solution works**, not merely obtaining the correct output.

Reminder

Your objective is to develop the ability to approach unfamiliar problems independently, translate your reasoning into Python code, and verify your solutions through testing.

Understand the problem first. Build the logic next. Code only after that.

How to Submit

1. Solve all 20 questions using Jupyter Notebook.
2. **Document your six-step problem-solving approach for every question as comments or markdown**.
3. Include the Python code and verify the output for each question.
4. Test every solution with at least one additional test case.
5. Save your completed notebook as an `.ipynb` file.
6. Upload your completed work **strictly through the LMS**.

Important: Task submissions will be strictly evaluated for certification purposes. Ensure that your work is complete, independently attempted, and submitted within the specified deadline.

Questions

Q1. Find the Shortest Word in a Sentence

Write a Python program to find the shortest word in a given sentence.

Example input:

Python makes coding easy

Expected output:

easy

Q2. Find the Largest Number in a List

Write a Python program to find the largest number in a list without using the built-in `max()` function.

Example input:

[12, 45, 7, 89, 23]

Expected output:

89

Q3. Find the Smallest Number in a List

Write a Python program to find the smallest number in a list without using the built-in `min()` function.

Example input:

[12, 45, 7, 89, 23]

Expected output:

7

Q4. Find the First Longest Word When Ties Occur

Write a Python program to find the first longest word in a sentence. If multiple words have the same maximum length, return the one that appears first.

Example input:

Python makes coding useful

Expected output:

Python

Q5. Find the Word with the Maximum Number of Characters

Write a Python program to identify the word containing the maximum number of characters in a sentence.

Example input:

Data engineering requires problem solving

Expected output:

engineering

Q6. Count Vowels in a String

Write a Python program to count the total number of vowels in a string. Consider both uppercase and lowercase vowels.

Example input:

Education

Expected output:

5

Q7. Count Consonants in a String

Write a Python program to count the consonants in a string. Ignore spaces, digits, and special characters.

Example input:

Python 123!

Expected output:

5

Q8. Find the Most Frequent Character

Write a Python program to find the character that appears most frequently in a string. If multiple characters have the same highest frequency, return the first one encountered. Ignore spaces.

Example input:

banana

Expected output:

a

Q9. Find the Least Frequent Character

Write a Python program to find the character that appears least frequently in a string. If multiple characters have the same lowest frequency, return the first one encountered. Ignore spaces.

Example input:

success

Expected output:

u

Q10. Find All Characters Appearing More Than Once

Write a Python program to identify all characters that appear more than once in a string. Display each repeated character only once, in the order of its first appearance. Ignore spaces.

Example input:

programming

Expected output:

['r', 'g', 'm']

Q11. Create a Dictionary of Words and Their First Characters

Write a Python program to create a dictionary in which each word is a key and its first character is the value.

Example input:

Python Data Engineering

Expected output:

{'Python': 'P', 'Data': 'D', 'Engineering': 'E'}

Q12. Create a Dictionary of Words and Their Last Characters

Write a Python program to create a dictionary in which each word is a key and its last character is the value.

Example input:

Python Data Engineering

Expected output:

{'Python': 'n', 'Data': 'a', 'Engineering': 'g'}

Q13. Create a Dictionary of Words and Their Uppercase Versions

Write a Python program to create a dictionary in which each word is a key and its uppercase version is the value.

Example input:

data science python

Expected output:

{'data': 'DATA', 'science': 'SCIENCE', 'python': 'PYTHON'}

Q14. Create a Dictionary of Numbers and Their Squares

Write a Python program to create a dictionary containing each number in a list as a key and its square as the value.

Example input:

[2, 3, 4, 5]

Expected output:

{2: 4, 3: 9, 4: 16, 5: 25}

Q15. Create a Dictionary of Numbers and Their Even/Odd Status

Write a Python program to create a dictionary containing each number as a key and `"Even"` or `"Odd"` as its value.

Example input:

[1, 2, 3, 4, 5]

Expected output:

{1: 'Odd', 2: 'Even', 3: 'Odd', 4: 'Even', 5: 'Odd'}

Q16. Find the Most Frequent Word in a Sentence

Write a Python program to find the word that appears most frequently in a sentence. Treat uppercase and lowercase versions of a word as the same word. If multiple words share the highest frequency, return the one that appears first.

Example input:

Python is easy and Python is powerful

Expected output:

python

Q17. Find All Words Appearing More Than Once

Write a Python program to find all words that appear more than once in a sentence. Display each repeated word only once, in the order of its first appearance. Treat uppercase and lowercase versions as the same word.

Example input:

data science uses data and science tools

Expected output:

['data', 'science']

Q18. Find the First Word That Appears Only Once

Write a Python program to find the first word that occurs exactly once in a sentence. Treat uppercase and lowercase versions as the same word.

Example input:

apple banana apple mango banana orange

Expected output:

mango

Q19. Count Words Whose Length Is Greater Than 4

Write a Python program to count the words containing more than four characters in a sentence.

Example input:

Python makes problem solving interesting

Expected output:

4

Q20. Find the Longest Word That Appears Only Once

Write a Python program to find the longest word that appears exactly once in a sentence. If multiple qualifying words have the same maximum length, return the first one encountered. Treat uppercase and lowercase versions as the same word.

Example input:

Python makes coding useful Python

Expected output:

coding